

Glossaire : tous les termes techniques du PowerPoint ColorRoom

Fiche de récap pour l'oral E6 · Téo Trompier (E2). Chaque terme = **définition simple** + *(où/pourquoi dans le projet)*. Lis-le une fois en entier, puis relis les sections 5, 6 et 11 (ce sont celles qui « impressionnent » le jury).

1. Mots de vocabulaire (les « mots savants »)

- **Paradigme** : une *façon de penser/d'organiser* le code. Ex. : Node-RED suit le paradigme **flux** (on relie des boîtes) ; React suit le paradigme **composants** (on assemble des briques d'interface).
 - **Flow-based / dataflow** : « basé sur le flux de données » : on programme en **reliant des nœuds** par des fils (la donnée circule de boîte en boîte). C'est le modèle de Node-RED.
 - **Déclaratif** : on **décrit le résultat voulu** (« je veux ce bouton ici »), pas les étapes pour l'obtenir. React est déclaratif. L'opposé = **impératif** (on écrit chaque étape).
 - **Diffable** : se dit d'un fichier dont on **voit clairement les différences** (le « diff ») entre deux versions. Du code texte est diffable ; un gros JSON de flux Node-RED ne l'est pas vraiment → difficile à relire dans Git.
 - **Versionnable** : qu'on peut suivre **version par version** dans un outil comme Git (historique, retour en arrière, qui a changé quoi).
 - **Idempotent** : une opération qu'on peut **rejouer plusieurs fois sans rien casser** ni dupliquer. Mes migrations de base sont idempotentes : relancer l'app ne recrée pas les tables existantes.
 - **Heuristique** : une **règle approximative** qui donne une « bonne » réponse vite, sans tout calculer. Mon IA note les positions avec une heuristique (au lieu d'explorer l'infini).
 - **Atomique** : **insécable** : l'opération se fait **entièrement ou pas du tout**, jamais à moitié.
 - **Concurrence** : plusieurs choses qui veulent s'exécuter **en même temps** (et qu'il faut coordonner).
 - **Volatile** : **temporaire, en mémoire vive** : perdu dès qu'on recharge/éteint (par opposition à *persistant*).
 - **Persistant** : **conservé durablement** (sur disque) : survit aux redémarrages.
-

2. Front-end · React / Next.js

- **Composant** : une **brique d'interface réutilisable** (un bouton, une carte, un nœud). On assemble l'UI avec des composants.
- **JSX / TSX** : la syntaxe qui mélange **HTML et code** dans un composant React. **TSX** = la version **TypeScript** (mes fichiers sont en `.tsx`).
- **DOM** : *Document Object Model* : la **représentation de la page** (l'arbre des éléments HTML) que le navigateur affiche.
- **Virtual DOM (DOM virtuel)** : une **copie légère du DOM en mémoire**. React compare l'ancienne et la nouvelle copie et ne met à jour **que ce qui a changé** → rapide.
- **Réconciliation** : le **calcul de la différence** entre l'ancien et le nouveau Virtual DOM, pour appliquer le minimum de changements à l'écran.
- **État (state) réactif** : des données qui, **quand elles changent, redessinent l'interface** automatiquement.
- **useState** : l'outil React pour une **variable réactive** (changer sa valeur ré-affiche le composant).
- **useRef** : une **variable mémoire qui ne déclenche pas de re-rendu** (état volatile rapide, ex. l'état des dalles pendant un jeu).
- **Next.js** : le **framework** qui réunit le front (React) **et** le back (API) dans un seul projet.

- **App Router** : le système de **pages et de routes** de Next.js (chaque dossier = une URL).
 - **Route Handler** : une **route d'API** (un fichier `route.ts` qui répond à une requête HTTP, ex. `/api/auth/login`).
 - **Server Component** : un composant **rendu côté serveur** (HTML prêt à l'emploi, plus léger pour le navigateur).
 - **SSR** (*Server-Side Rendering*) : **rendu de la page côté serveur** avant de l'envoyer au navigateur.
 - **Three.js** : la **bibliothèque 3D** (WebGL) qui dessine la salle ColorRoom en 3D.
 - **WebGL / WebGLRenderer** : la techno qui affiche de la **3D dans le navigateur** (via la carte graphique).
-

3. TypeScript · qualité du code

- **TypeScript** : du JavaScript **avec des types** (on précise qu'une variable est un nombre, un texte...).
 - **Typage statique** : les types sont **vérifiés avant l'exécution** (à la compilation), pas pendant.
 - **Compilation / transpilation** : **traduire** le TypeScript en JavaScript exécutable ; au passage, on détecte les erreurs.
 - `tsc` : le **compilateur TypeScript** (vérifie les types).
 - **ESLint** : un **analyseur** qui repère erreurs et mauvaises pratiques dans le code.
 - **Runtime** : l'**environnement d'exécution** (ici **Node.js**) ; « au runtime » = « pendant que ça tourne ».
-

4. Base de données · SQLite

- **SQLite** : une base de données **embarquée** : un **simple fichier**, sans serveur à installer. Idéale sur Raspberry Pi.
 - **better-sqlite3** : la **bibliothèque** Node qui pilote SQLite, en **synchrone** (résultat immédiat, code plus simple).
 - **Requête préparée** : une requête SQL avec des **paramètres** `?` ; les valeurs sont injectées séparément → **anti-injection SQL**.
 - **Injection SQL** : une **attaque** où l'on glisse du SQL malveillant dans un champ ; bloquée par les requêtes préparées.
 - **WAL** (*Write-Ahead Logging*) : un mode SQLite qui permet de **lire pendant qu'on écrit** (lectures concurrentes) → moins de blocages.
 - `busy_timeout` : délai pendant lequel SQLite **réessaie** au lieu de renvoyer une erreur « base occupée » (`SQLITE_BUSY`).
 - **Clé étrangère (foreign key)** : un lien entre tables (ex. un score appartient à un user) ; **ON DELETE CASCADE** = supprimer un user supprime ses scores.
 - **Migration** : la **création/mise à jour du schéma** (les tables) au démarrage ; les miennes sont **idempotentes**.
 - **Singleton** : une **seule instance** partagée (ici, une seule connexion à la base pour toute l'app).
 - **ERD** (*Entity-Relationship Diagram*) : le **schéma des tables** et de leurs relations.
-

5. Variables & persistance (les questions « pointues »)

- **Transaction** : un **groupe d'opérations traité comme un seul bloc**.
- **ACID** : les 4 garanties d'une transaction : **Atomicité** (tout ou rien), **Cohérence**, **Isolation**, **Durabilité**.

- **Variable transactionnelle** : une opération encadrée par `BEGIN ... COMMIT` (et `ROLLBACK` si erreur) : l'inscription crée l'utilisateur **et** son adhésion de classe en une seule unité : **tout ou rien**.
- **ROLLBACK** : **annulation** de toutes les écritures si une étape échoue (pas de compte « à moitié créé »).
- **Variable atomique** : une valeur partagée modifiée de façon **insécable**. Ici `hwInFlight` (le nombre d'accès matériel en cours) : il est atomique **car Node est mono-thread**.
- **Sémaphore** : un **compteur qui limite** le nombre d'accès simultanés à une ressource (ici **2** accès max au matériel).
- **Mono-thread** : Node exécute le code sur **un seul fil** : deux lignes ne tournent jamais **vraiment** en même temps → pas de conflit sur une variable simple.
- **Boucle d'événements (event loop)** : le mécanisme de Node qui **enchaîne les tâches** une par une (et gère l'asynchrone).
- **Promise** : une **valeur future** (résultat d'une opération asynchrone : « ce sera prêt plus tard »).
- **Promise.all** : lance **plusieurs opérations en parallèle** et attend qu'elles finissent toutes (ex. envoyer les 32 canaux d'une dalle).
- **File d'attente (queue)** : les requêtes en trop **patientent** jusqu'à ce qu'un créneau se libère.
- **Variable d'environnement** : un réglage **hors du code**, lu via `process.env` depuis le fichier `.env` (clé API, mot de passe admin) : **jamais commité dans Git**.

6. Sécurité · RGPD

- **Hachage** : transformer un mot de passe en une **empreinte irréversible** (on ne peut pas « déhacher »). On ne stocke jamais le mot de passe en clair.
- **PBKDF2** (*Password-Based Key Derivation Function 2*) : un algorithme de hachage **volontairement lent** (par répétitions) pour résister aux attaques.
- **HMAC** : une fonction de hachage **avec une clé secrète** ; **SHA-512** est la fonction de base utilisée.
- **Itérations** : le nombre de répétitions du calcul (**100 000** ici) : plus c'est élevé, plus c'est **coûteux à casser**.
- **Sel (salt)** : une **valeur aléatoire unique** ajoutée à chaque mot de passe avant hachage → deux mêmes mots de passe donnent des empreintes différentes.
- **Rainbow table** : une **base d'empreintes pré-calculées** utilisée par les attaquants ; le **sel** la rend inutile.
- **Cookie** : un petit fichier déposé chez le navigateur pour **reconnaître** l'utilisateur connecté (porte le jeton de session).
- **HttpOnly** : cookie **inaccessible au JavaScript** → protège du vol par script (**anti-XSS**).
- **SameSite=lax** : le cookie n'est **pas envoyé depuis un autre site** → protège du **CSRF**.
- **XSS** (*Cross-Site Scripting*) : injection de **JavaScript malveillant** dans la page.
- **CSRF** (*Cross-Site Request Forgery*) : faire exécuter une **action à l'insu** de l'utilisateur connecté.
- **Session / jeton (token)** : un **identifiant aléatoire** (`randomBytes`) qui prouve qu'on est connecté ; valable **30 jours** avec **renouvellement glissant**.
- **RGPD** : la réglementation sur les **données personnelles** : ici **minimisation** (un simple pseudo suffit), mots de passe hachés, **droit à l'effacement** (suppression en cascade), tout **local**.

7. Intelligence artificielle (Puissance 4)

- **Minimax** : algorithme de jeu : on **simule l'arbre des coups** ; l'IA cherche à **MAXimiser** son score, l'adversaire à le **MINimiser**. L'IA suppose que l'adversaire joue au mieux.
- **Élagage alpha-bêta** : une **optimisation** du minimax qui **coupe les branches inutiles** (celles qui ne changeront pas la décision) → l'IA explore **plus profond, plus vite**.

- **Profondeur** : le **nombre de coups anticipés** (1, 2, 5, 9, 12 selon le niveau Novice → Légendaire).
 - **Fenêtre de 4** : chaque **alignement possible de 4 cases** ; l'IA note la position en additionnant le score de toutes les fenêtres.
 - **Centralité** : bonus donné aux **colonnes centrales** (plus stratégiques au Puissance 4).
 - **Anti-piège** : l'IA **ne joue pas un coup qui offre la victoire immédiate** à l'adversaire ; la **défense est pondérée plus fort que l'attaque** (-170 contre +130).
-

8. Réseau · multijoueur

- **Hors-ligne (offline)** : l'app fonctionne **sans Internet** : tout est local (Pi + SQLite + matériel).
 - **Wi-Fi local** : le Pi diffuse son propre réseau **ColorRoom_WiFi** ; les tablettes s'y connectent.
 - **Proxy** : un **intermédiaire** : le navigateur ne parle pas au matériel directement, il passe par une **route relais** côté serveur.
 - **WebSocket** : une connexion **temps réel bidirectionnelle** (non utilisée ici, volontairement).
 - **Polling** : au lieu d'un WebSocket, chaque client **interroge régulièrement** `/state` pour récupérer l'état → plus simple et robuste sur Pi.
 - **Heartbeat** : un **signal de présence** envoyé régulièrement pour savoir qui est encore connecté.
 - **UUID** : un **identifiant unique** universel (pour les sessions/joueurs).
 - `state_json` : l'état de la partie multijoueur **sérialisé** (texte JSON) et **stocké en base** (`crg_mp_sessions`).
 - **AbortController** : permet d'**annuler une requête** trop longue (**timeout**) pour ne pas bloquer le matériel.
-

9. Colorimétrie · matériel (la ColorRoom)

- **RS-485** : un **bus de communication série** (différentiel, multi-points, robuste sur longue distance) qui pilote les **42 plaques LED**.
 - **Canal (LED)** : une **commande de couleur** ; chaque plaque a **32 canaux = 24 spectres étroits** (proche UV → proche IR) + **8 blancs**.
 - **Spectre** : la **répartition de la lumière par longueur d'onde** (la « recette » d'une couleur).
 - **Longueur d'onde** : mesurée en **nanomètres (nm)** ; détermine la couleur perçue ($\approx$ 400 nm violet → 780 nm rouge).
 - **Colorimètre CS-150/160** : l'appareil **Konica Minolta** qui **mesure** la lumière émise.
 - **Tristimulus XYZ (CIE)** : **3 valeurs** qui décrivent une couleur telle que **perçue par l'œil** (base de la colorimétrie).
 - **CIE 1931** : le **diagramme** standard (« fer à cheval ») où l'on place une couleur par ses coordonnées.
 - **Chromaticité (x, y)** : la **couleur sans la luminosité** (sa position sur le diagramme CIE).
 - **Luminance L_v** : l'**intensité lumineuse perçue**, en **cd/m²** (candela par mètre carré).
 - **ΔE (Delta E)** : l'**écart perçu entre deux couleurs** ; sert à scorer la précision dans les jeux de mesure.
 - **Remappage des canaux** : convertir une couleur **RGB** demandée vers les **32 canaux** réels de la plaque (via les profils par longueur d'onde).
 - **Pont .NET** : le **petit programme** qui fait le lien entre mon API et le colorimètre.
-

10. Infrastructure · DevOps

- **Docker** : outil qui **emballe l'app et ses dépendances** dans une **image** ; on lance cette image en **conteneur** (« ça marche pareil partout »).
 - **Conteneur** : une **instance qui tourne** à partir d'une image Docker.
 - **Image multi-stage** : un **Dockerfile en plusieurs étapes** (deps → build → runner) pour une image finale plus légère.
 - **arm64** : l'**architecture processeur** du Raspberry Pi (l'image Docker est compilée pour elle).
 - **Volume** : un **dossier persistant** monté dans le conteneur (la base SQLite y survit aux redémarrages).
 - **Portainer** : une **interface web** pour superviser les conteneurs Docker (logs, redémarrage).
 - **CI/CD : Intégration / Déploiement Continu** : à chaque envoi de code, une chaîne **teste, build et déploie** automatiquement (ici sur le Pi via GitLab).
 - **Git / commit / branche** : l'outil de **gestion de versions** ; un ***commit*** = une sauvegarde datée ; une ***branche*** = une ligne de travail parallèle.
 - **CSV · UTF-8 · BOM** : format d'**export tableur** ; **UTF-8** = encodage des accents ; **BOM** = marqueur qui fait que **Excel ouvre bien les accents**.
 - **Ollama** : un moteur d'**IA locale** (hors-ligne) ; **Gemini** = l'IA cloud optionnelle.
-

11. UML (les diagrammes)

- **UML** : un **langage de schémas** standard pour décrire un logiciel.
 - **Cas d'utilisation** : qui (**acteurs**) fait quoi avec le système (Enseignant, Apprenant).
 - **Diagramme de classes** : la **conception objet** (les entités et leurs liens, côté code).
 - **Diagramme de composants** : les **grands blocs** logiciels et leurs échanges.
 - **Diagramme de déploiement** : le **matériel réel** et ce qui tourne dessus (Pi, Docker, dalles...).
 - **Diagramme de séquence** : le **dialogue chronologique** entre éléments (qui appelle qui, dans l'ordre).
 - **Diagramme d'états** : les **états** d'un objet et les **transitions** (ex. partie : prête → en cours → terminée).
 - **Diagramme d'activité** : un **enchaînement d'actions** (un « organigramme »), ex. le remappage des canaux.
 - **Gantt** : le **planning** du projet (tâches dans le temps, par étudiant).
-

12. Les 4 extraits de code expliqués

Tous tirés du dépôt réel [NewGenesisAgency/color_room](#) (branche [main](#)).

A. Variable transactionnelle (ACID)

Fichier : [.../main/app/app/api/auth/register/route.ts#L47-L62](#)

Ce que ça fait : `db.transaction(() => { ... })` regroupe **deux écritures** : créer l'utilisateur **et** (si un code de classe est fourni) l'inscrire à sa classe. C'est **tout-ou-rien** : si la 2^e écriture échoue, la 1^{re} est **annulée (ROLLBACK)** → jamais de compte « à moitié créé ». C'est l'**Atomicité** du « A » d'ACID.

À dire : « J'utilise une **variable transactionnelle** : un `BEGIN ... COMMIT` automatique, avec **ROLLBACK** en cas d'erreur. »

B. Variable atomique · sémaphore matériel

Fichier : [.../main/app/app/api/supervision/batch/route.ts#L39-L80](#)

Ce que ça fait : `hwInFlight` compte les **accès au matériel en cours**. `acquireHwSlot()` n'autorise que **2 accès simultanés** (`HW_CONCURRENCY`) ; au-delà, les requêtes attendent dans une **file de Promises**, réveillées par `releaseHwSlot()` . C'est un **sémaphore**. La variable est **atomique** parce que Node est **mono-thread** (pas de modification simultanée réelle).

À dire : « `supervision.exe` est **quasi-série** ; je **borne la concurrence** avec un compteur atomique pour ne pas le saturer. »

C. Hachage des mots de passe (PBKDF2)

Fichier : `.../main/app/lib/auth.ts#L19-L23`

Ce que ça fait : `pbkdf2Sync(password, salt, 100_000, 64, 'sha512')` calcule une **empreinte** du mot de passe avec un **sel aléatoire** (16 o), **100 000 itérations**, sortie **64 o**, fonction **SHA-512**. On stocke `sel:hash` : **jamais** le mot de passe en clair. La vérification **recalcule** et compare.

À dire : « Hachage **PBKDF2-HMAC-SHA512**, lent et salé → résistant aux **rainbow tables** et au brute-force. »

D. Évaluation minimax (alpha-bêta)

Fichier : `.../main/app/app/_components/GamePuissance4.tsx#L118-L129`

Ce que ça fait : `scoreWindow(me, opp)` note une **fenêtre de 4 cases** du point de vue de l'IA : victoire = `WIN_SCORE` (1 000 000), 3 alignés à moi = **+130**, 3 alignés adverses = **-170** (la **défense prime**). Ces scores nourrissent le **minimax + élagage alpha-bêta** qui choisit le meilleur coup.

À dire : « Heuristique par **fenêtres de 4**, **défense pondérée plus fort que l'attaque**, le tout dans un **minimax alpha-bêta** à profondeur variable. »

*Astuce d'oral : si le jury demande « c'est quoi X ? », donne la définition simple **puis** un exemple tiré du projet. C'est ce qui montre que tu maîtrises.*

</content>